

Solutions d'optimisation intégrées à la base de données

Par

Étienne Toupin, Mathieu Fortier, Mikaël Lachaine, Imène Kara Mostefa, Mikaël Demers, Antoine-Oliver Monaco

Travail présenté à

Luc Lavoie

Dans le cadre du cours

IGE487 - Modélisation de bases de données

Université de Sherbrooke

14 novembre 2025

Introduction

Ce rapport présente une analyse approfondie des performances de la base de données Herbivorie, centrée sur l'optimisation des requêtes les plus critiques liées au suivi écologique des plants de trille. Les trois optimisations successives menées sur ce système révèlent des patterns cohérents en matière de stratégie d'indexation et fournissent des recommandations actionnables pour l'amélioration des performances. Ensemble, ces trois optimisations mettent en évidence un principe fondamental : l'efficacité des index est directement corrélée à la sélectivité des requêtes. Les améliorations significatives observées pour les plages de dates restreintes contrastent avec les gains marginaux, voire négatifs, pour les analyses exhaustives. Cette compréhension fine du comportement du système permet d'établir une stratégie d'indexation équilibrée, ciblant spécifiquement les cas d'usage opérationnels les plus fréquents tout en évitant la sur-indexation contre-productive.

Optimisation 1

Requête: Combien de plants par placette / par zone / dans une plage de dates ?

Objectif: Mesurer l'impact de l'indexation sur les placettes et les zones selon une plage de date déterminée.

Jeux de test

Répertoire: Optimisation_test

- TEST #1: plage de date sélective -> 2017-06-01 → 2017-06-15 (2 semaines)
 - Voir fichier TEST1_plant_par_placette_par_zone_selon_dates.log
- TEST #2: plage de date large: 2017-05-01 → 2017-09-01 (couvre quasiment tous mes date)
 - Voir fichier TEST2_plant_par_placette_par_zone_selon_dates.log

Résumé rapide

Agrégat	Plage	Avant index	Après index	Gain constaté
Par placette	Sélective (Test #1)	~16.95 ms	~0.91 ms	18x plus rapide
Par zone	Sélective (Test #1)	~15.19 ms	~1.15 ms	13x plus rapide

Par placette	Large (Test #2)	~20.97 ms	~23.76 ms	0.9x moins rapide
Par zone	Large (Test #2)	~20.94 ms	~18.63 ms	1.1x plus rapide

Explication

Avec une **plage de date courte**, on aperçoit que l'index est gagnant. Il est beaucoup plus efficace en termes de recherche et de tri. Grâce à l'index sur plant, il peut directement choisir les dates des plants qui sont concernés et mettre de côté les autres, donc la requête est beaucoup plus rapide.

De l'autre côté, si la **plage de date couvre quasiment tous les dates** de ma base de donnée, l'index ne sert à rien, car tous les lignes vont passer dans le filtre, on est obligé de les passer en séquentiel alors le temps est pratiquement le même, sauf dans les placettes, le temps semble moins rapide. Peu de gain est fait dans une plage de très grande date.

Conclusion

Un index sur Plant(date, parcelle_id) est très utilisé pour une plage de date très courte, car on évite de parcourir toute la table et accélère énormément la requête.

À l'inverse, pour des très grandes plages, l'index n'apporte aucun gain, un scan séquentiel sera fait. Bref, très utilisé pour des plages de dates quand même bien rapprochées et peut être très efficace sur un BD qui contient énormément de données, la mienne en contenant entre 300 et 500.

Optimisation 2

Requête analysée

Évolution moyenne des dimensions des plants par placette sur une période

```
SELECT p.placette_id,
       AVG(od.longueur) as moyenne_longueur,
       AVG(od.largeur) as moyenne_largeur,
       COUNT(od.id) as nb_observations
FROM ObsDimension od
JOIN Plant pl ON od.id = pl.id
JOIN Parcelle p ON pl.parcelle_id = p.parcelle_id
WHERE od.date BETWEEN ? AND ?
```

```
GROUP BY p.placette_id;
```

Jeux de Test

Répertoire : Optimisation_test

TEST #1 : Période courte → 2017-06-01 → 2017-06-15 (2 semaines)

Voir fichier *TEST1_distrubution_plant_placette_courte.log*

TEST #2 : Période longue → 2017-05-01 → 2017-09-01 (4 mois)

Voir fichier *TEST2_dimension_par_placette_longue.log*

Résumé des Performances

Métrique	Plage	Avant Index	Après Index	Gain Constaté
Temps moyen	Sélective	12.661 ms	5.127 ms	2.5x plus rapide
Temps moyen	Large	19.393 ms	18.175 ms	1.1x plus rapide

Index implémenté

```
CREATE INDEX idx_obsdimension_date_plant ON ObsDimension(date, id);
```

Explication

Pour la période courte (Test #1), il y a une grande amélioration dans le temps d'exécution. L'index sur ObsDimension(date,id) permet une sélection rapide des observations dans la plage de dates. Seules les lignes pertinentes sont accédées via l'index. Pour la période longue (Test #2), l'optimiseur choisit le scan séquentiel car la plage couvre 85% des données. Le coût d'accès à l'index et l'accès à la table peut dépasser celui du scan complet. Les jointures bénéficient tout de même légèrement des index secondaires.

Conclusion

L'index sur ObsDimension (date, id) est extrêmement efficace pour les requêtes analytiques sur des périodes restreintes, les analyses saisonnières ou mensuelles et les rapports ciblés sur des événements spécifiques. Cependant, cela peut être peu

utile pour les analyses annuelles complètes. L'optimiseur préfère le scan séquentiel pour les grandes plages.

Optimisation 3

Requête analysée

Distribution des états des plants par zone écologique

```
SELECT z.code_zone, oe.etat, COUNT(*) as nb_plants
FROM ObsEtat oe
JOIN Plant pl ON oe.id = pl.id
JOIN Parcelle p ON pl.parcelle_id = p.parcelle_id
JOIN Placette plct ON p.placette_id = plct.plac
JOIN Zone z ON plct.zone = z.code_zone
WHERE oe.date BETWEEN ? AND ?
GROUP BY z.code_zone, oe.etat;
```

Jeux de Test

Répertoire : Optimisation_test

TEST #1 : Période courte → 2017-06-01 → 2017-06-15 (2 semaines)

Voir *fichier TEST1_distribution_par_placette_courte.log* et
TEST2_distribution_par_placette_courte.log

TEST #2 : Période longue → 2017-05-01 → 2017-09-01 (4 mois)

Voir *fichier TEST1_dimension_par_placette_longue.log*

Résumé des Performances

Métrique	Plage	Avant Index	Après Index	Gain Constaté
Temps moyen	Sélective	19.434 ms	2.127 ms	9x plus rapide
Temps moyen	Large	14.029 ms	6.082 ms	2.3x plus rapide

Index implémenté

```
CREATE INDEX idx_obsetat_date_etat_plant ON ObsEtat(date, etat, id);
```

```
CREATE INDEX idx_placette_zone ON Placette(zone);
```

Explication

Pour une période courte de deux semaines, l'index composite sur ObsEtat(date, etat, id) permet une amélioration des performances, réduisant le temps d'exécution. Cet index permet à l'optimiseur de filtrer rapidement les observations par date tout en ayant accès directement aux états et identifiants des plants nécessaires aux jointures et regroupements. En revanche, pour une période large de quatre mois couvrant la majorité des données, le gain devient négligeable avec même un léger ralentissement. Dans ce scénario, l'optimiseur PostgreSQL préfère effectuer un scan séquentiel complet de la table plutôt que d'utiliser l'index, car le coût d'accès indexé à presque toutes les lignes dépasse celui du parcours complet. L'index sur Placette(zone) améliore quant à lui légèrement les jointures dans les deux cas, mais son impact reste secondaire comparé à la sélectivité temporelle.

Conclusion

Les index sur les dates sont très efficaces pour les requêtes sélectives mais apportent moins pour les analyses globales. La stratégie d'indexation doit cibler les cas d'usage opérationnels les plus fréquents.